```
#include<time.h>
main()
{
 int arr[10000],i,j,min,temp;
 for(i=0;i<10000;i++)
  {
    arr[i]=rand()%10000;
  }
 //The  MySort Agorithm
 clock_t start,end;
 start=clock();

min=arr[0];
int max=arr[0] ;
for(i=1;i<10000;i++)
 {
  if(arr[i]<min)
   min=arr[i] ;     // required for optimization
  else if(arr[i]>max)
   max=arr[i] ;
 }
int *hash=(int*)calloc((max+1), sizeof(int)); //should be max+1

for(i=0;i<10000;i++ )
 {
  hash[arr[i]]++;  // storing the count of occurances of elements
 }
 printf("\n");
 end=clock();
 double extime=(double) (end-start)/CLOCKS_PER_SEC;
 printf("\n\tExecution time for the  MySort Algorithm is %f
seconds\n ",extime);


 for(i=0;i<10000;i++)
 {
  arr[i]=rand()%10000;
 }
 clock_t start1,end1;
 start1=clock();
 // The Selection Sort
 for(i=0;i<10000;i++)
  {
   min=i;
   for(j=i+1;j<10000;j++)
    {
     if(arr[min]>arr[j])
     {
      min=j;
     }
    }
   temp=arr[min];
   arr[min]=arr[i];
   arr[i]=temp;
 }
 end1=clock();
 double extime1=(double) (end1-start1)/CLOCKS_PER_SEC;
 printf("\n");
 printf("\tExecution time for the selection sort is %f
seconds\n \n",extime1);
 if(extime1<extime)
  printf("\tSelection sort is faster than MySort Algorithm
by %f seconds\n\n",extime-    extime1);
 else if(extime1>extime)
  printf("\tMysort Algorithm is faster than  Selectionsort
by %f seconds\n\n",extime1-extime);
 else
  printf("\tBoth algo has same execution time\n\n");
 }
```

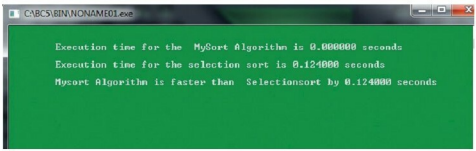

Figure 1: Result -> speed up difference between two Sorting Algorithm

Note: It is not always recommended to use *clock ()* since it is badly [not precise] implemented on many architectures. Precise optimisation can only be done by analysing the code. Asymptotic analysis is the perfect way for analysing algorithms. It evaluates the performance of an algorithm in terms of input size [we don't measure the actual running time].

How is this sorting algorithm better than Merge sort and Quick sort algorithms in terms of both time complexity and overhead spent?

Both Merge sort [O(nlogn)] and Quick sort [O(nlogn)-best case] use recursion to sort the input; also, the running time of Quick sort in the worst case is $O(n^2)$. There are always certain overheads involved while calling a function. Time has to be spent on passing values, passing control, returning values and returning control. Recursion takes a lot of stack space. Every time the function calls itself, memory has to be allocated, and also recursion is not good for production code.

In the new sorting algorithm, we are getting rid of all issues—the recursion, the time and the overhead. In comparison with the Merge and Quick sort, we are almost cutting the time by logn and making the new algorithm to run in O(n) time.

It may happen that when you try to compare the execution time of Merge and Quick sort and the new sorting algorithm, they may give the same or equal execution time based on the above method by introducing *clock*, but again, you cannot